

1. Practical5.c

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

CODE:

```
GNU nano 8.7.1 equivalent.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pipe(fd); // Create a pipe

    if (fork() == 0) {
        // Child 1: executes "ls -l"
        dup2(fd[1], STDOUT_FILENO); // Redirect stdout to pipe write end
        close(fd[0]);
        close(fd[1]);
        execlp("ls", "ls", "-l", NULL);
    }

    if (fork() == 0) {
        // Child 2: executes "grep .c"
        dup2(fd[0], STDIN_FILENO); // Redirect stdin to pipe read end
        close(fd[0]);
        close(fd[1]);
        execlp("grep", "grep", ".c", NULL);
    }

    // Parent closes both ends and waits
    close(fd[0]);
    close(fd[1]);
    wait(NULL);
    wait(NULL);

    return 0;
}
```

```
advaith@DESKTOP-0E1OBMR:~/PRACTICAL/my_shell$ gcc equivalent.c -o
advaith@DESKTOP-0E1OBMR:~/PRACTICAL/my_shell$ ./program
-rw-r--r-- 1 advaith advaith  703 Oct  5 03:51 equivalent.c
-rwxr-xr-x 1 advaith advaith 16432 Oct  5 03:46 my_shell
-rwxr-xr-x 1 advaith advaith 16216 Oct  5 03:53 program
-rw-r--r-- 1 advaith advaith  1061 Oct  5 03:46 shell.c
advaith@DESKTOP-0E1OBMR:~/PRACTICAL/my_shell$ |
```

OUTPUT:

6. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

CODE:

```
GNU nano 8.7.1 signaldemo.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Signal handler function
void handle_signal(int sig) {
    switch(sig) {
        case SIGINT:
            printf("\nCaught SIGINT (Ctrl+C). Ignoring...\n");
            break;
        case SIGTERM:
            printf("\nCaught SIGTERM. Terminating gracefully...\n");
            exit(0);
            break;
        case SIGUSR1:
            printf("\nCaught SIGUSR1. Custom user signal received!\n");
            break;
        default:
            printf("\nCaught unexpected signal: %d\n", sig);
    }
}

int main() {
    // Register signal handlers
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Process running with PID: %d\n", getpid());
    printf("Try sending signals using:\n");
    printf("  kill -SIGUSR1 %d\n", getpid());
    printf("  kill -SIGTERM %d\n", getpid());
    printf("Press Ctrl+C to send SIGINT.\n", getpid());

    // Infinite loop to keep process alive
    while (1) {
        printf("Waiting for signals...\n");
        sleep(5);
    }

    return 0;
}
```

Ex: kill -SIGUSR1 PID

Ex: kill -SIGTERM PID

OUTPUT:

```
advaith@DESKTOP-0E10BMR:~/PRACTICAL/my_shell$ ./^C
advaith@DESKTOP-0E10BMR:~/PRACTICAL/my_shell$ nano signaldemo.c
advaith@DESKTOP-0E10BMR:~/PRACTICAL/my_shell$ gcc signaldemo.c -o program
advaith@DESKTOP-0E10BMR:~/PRACTICAL/my_shell$ ./program
Process running with PID: 855
Try sending signals using:
    kill -SIGUSR1 855
    kill -SIGTERM 855
Press Ctrl+C to send SIGINT. (PID: 855)
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
Waiting for signals...
^C
Caught SIGINT (Ctrl+C). Ignoring...
Waiting for signals...
Waiting for signals...
^C
Caught SIGINT (Ctrl+C). Ignoring...
Waiting for signals...
Waiting for signals...
```